

Seguridad — Sistema de gestión ganadera

Propósito de este documento

Este documento define las decisiones, controles y requisitos de seguridad que el sistema debe implementar desde el día uno. No es una auditoría posterior ni un checklist de cumplimiento — es una guía de diseño que influye directamente en cómo se escribe el código, cómo se configura la infraestructura y cómo se validan los datos.

El sistema maneja información sensible de negocios privados: inventarios ganaderos, precios de compra y venta, resultados económicos por ejercicio. Una filtración de datos entre establecimientos no es solo un bug técnico — es una violación de confianza que destruye el producto.

Modelo de amenazas

Antes de definir controles hay que entender contra qué se protege el sistema. Para este contexto específico las amenazas relevantes son las siguientes.

Filtración de datos entre tenants. Es la amenaza más crítica. Un usuario de un establecimiento no debe poder ver ni inferir datos de otro. Esto incluye acceso directo (consultar endpoints ajenos) e indirecto (inferir IDs secuenciales, timing attacks, mensajes de error que revelan existencia de recursos).

Escalada de privilegios. Un usuario con rol veterinario no debe poder registrar movimientos. Un propietario no debe poder modificar datos. Un administrador no debe poder acceder a datos de otros establecimientos aunque manipule el token.

Manipulación de datos históricos. Los eventos son inmutables por diseño de negocio. Un atacante no debe poder editar o borrar eventos aunque tenga acceso legítimo al sistema.

Inyección y manipulación de inputs. Datos maliciosos en formularios, CSVs de importación, parámetros de búsqueda y filtros que intenten acceder a datos fuera del scope del usuario o ejecutar código en el servidor.

Exposición de tokens y credenciales. Tokens JWT filtrados en logs, variables de entorno expuestas en el frontend, claves de API en el repositorio.

Denegación de servicio accidental o intencional. Importaciones CSV masivas, queries sin paginación, endpoints que agregan datos sin límite.

Lo que no es una amenaza prioritaria en este contexto: ataques de estado-nación, ingeniería social avanzada, ataques físicos a infraestructura. El sistema no maneja datos financieros regulados ni información de salud personal.

Capa 1 — Base de datos

Esta es la capa más importante. Si la base de datos está bien configurada, un bug en el backend no puede filtrar datos entre tenants.

Row Level Security como primera línea de defensa.

RLS está habilitado en todas las tablas de dominio desde el día uno, como está definido en el esquema. La función `mis_establecimientos()` filtra automáticamente cada query al establecimiento del usuario autenticado. Esto significa que incluso si un endpoint del backend tiene un bug y no filtra por `establecimiento_id`, la base de datos rechaza el acceso.

La regla operativa es: nunca deshabilitar RLS en ninguna tabla de dominio, ni siquiera temporalmente para depuración. Si necesitas consultar datos sin RLS en desarrollo, usá la consola de Supabase con el rol de servicio, nunca desde el código de la aplicación.

Inmutabilidad de eventos a nivel de motor.

La regla `CREATE RULE eventos_no_update` definida en el esquema rechaza cualquier `UPDATE` sobre la tabla `eventos` a nivel de base de datos. Esto implementa RN-15 de forma que no puede ser ignorada desde el código. Ningún bug, ningún endpoint mal escrito y ningún desarrollador descuidado puede editar un evento porque el motor simplemente no lo permite.

La misma lógica aplica a los animales: no hay `DELETE` disponible en ninguna tabla de dominio. Las políticas RLS solo permiten `SELECT`, `INSERT` y `UPDATE` controlado. El borrado físico no existe como operación posible.

UUIDs en todos los identificadores.

Como está definido en las convenciones del esquema, todos los IDs son UUID v4. Esto elimina la posibilidad de enumerar recursos: un atacante no puede probar IDs secuenciales (1, 2, 3...) para acceder a registros de otros establecimientos. Incluso si RLS fallara en algún caso edge, los IDs no son predecibles.

Campos monetarios como NUMERIC, nunca FLOAT.

Definido en las convenciones del esquema. No es solo precisión aritmética — un sistema financiero con errores de redondeo en `FLOAT` puede ser manipulado para generar inconsistencias contables.

Capa 2 — Autenticación

JWT emitido y validado por Supabase Auth.

El backend FastAPI valida el JWT en cada request usando la clave pública de Supabase. Nunca confía en datos del cliente que no estén firmados en el token. El `establecimiento_id` del usuario se extrae del JWT validado, no de un parámetro del request.

Esto es crítico: si el `establecimiento_id` viniera como parámetro del request, un atacante autenticado podría pasar el ID de otro establecimiento. Al extraerlo del token firmado, eso es imposible.

Refresh tokens con rotación.

Supabase Auth maneja refresh tokens con rotación automática. Cuando un refresh token se usa, se invalida y se emite uno nuevo. Si un token es robado y usado por un atacante, el uso legítimo siguiente del usuario original invalida la sesión del atacante.

Expiración de sesión.

Los access tokens de Supabase expiran en una hora por defecto. El frontend maneja el refresh automáticamente con el SDK de Supabase. Las sesiones inactivas por más de 30 días deben requerir login nuevamente — configurable en el dashboard de Supabase.

No hay recuperación de contraseña por pregunta secreta.

Solo recuperación por email con link de un solo uso y expiración de 15 minutos. Supabase Auth lo implementa así por defecto.

Capa 3 — Backend

Validación en cada capa, no solo en la entrada.

El patrón de seguridad es *defense in depth*: el router valida que el usuario esté autenticado y tenga el rol correcto, el service valida que la operación sea coherente con las reglas de negocio, el crud valida que los datos sean del establecimiento correcto antes de escribir. Las tres capas validan independientemente. Una falla en una no compromete las otras.

Extracción del `establecimiento_id` desde el token, nunca desde el request.

Cada endpoint que accede a datos de dominio extrae el `establecimiento_id` del JWT validado. El patrón en FastAPI es el siguiente:

```
python
async def get_current_establecimiento_id(
    current_user: User = Depends(get_current_user)
) -> UUID:
    return current_user.establecimiento_id
```

Este valor se pasa a todos los servicios y operaciones de crud. Ningún endpoint acepta establecimiento_id como parámetro externo.

Validación de permisos por rol antes de cualquier operación de escritura.

El sistema implementa tres roles con permisos distintos. La validación ocurre en el router antes de llamar al service:

El propietario puede ejecutar únicamente operaciones de lectura: GET en todos los módulos, generación de reportes, exportación. Cualquier intento de POST, PUT o DELETE retorna 403.

El administrador puede ejecutar todas las operaciones dentro de su establecimiento. No puede acceder a datos de otros establecimientos.

El veterinario puede ejecutar operaciones de lectura en todos los módulos y operaciones de escritura únicamente en los módulos de sanidad: vacunaciones, tratamientos y diagnósticos. Cualquier intento de escritura en movimientos, pesajes o configuración retorna 403.

Paginación obligatoria en todos los endpoints de lista.

Ningún endpoint devuelve una lista sin límite. El límite máximo por página es 100 registros. El parámetro de límite viene con un default de 20 y un máximo hardcodeado en el servidor — el cliente no puede solicitar más de 100 registros en una sola llamada. Esto previene tanto la denegación de servicio como la exfiltración masiva de datos.

Rate limiting en endpoints críticos.

Los endpoints de autenticación (login, registro, recuperación de contraseña) tienen rate limiting de 10 requests por minuto por IP. Los endpoints de importación CSV tienen rate limiting de 5 requests por hora por usuario. Cloud Run maneja el rate limiting a nivel de infraestructura con Cloud Armor en V2 — en el MVP se implementa con slowapi en FastAPI.

Validación estricta de archivos CSV en importación.

La importación masiva es la superficie de ataque más amplia del sistema. Las reglas son: tamaño máximo de archivo de 5 MB, extensión verificada por content-type real no por nombre de archivo, número máximo de filas de 5000 por importación, cada campo validado contra el esquema Pydantic antes de escribir en la base, el procesamiento ocurre en un contexto aislado que no puede ejecutar código arbitrario.

Manejo de errores sin revelar internals.

Los errores que llegan al cliente nunca incluyen stack traces, nombres de tablas, queries SQL ni mensajes de la base de datos. El backend registra el error completo en los logs (Cloud Logging) y devuelve al cliente un mensaje genérico con un código de error referenciable. El mensaje "animal not found" cuando el animal existe pero pertenece a otro establecimiento es igual al mensaje cuando el animal realmente no existe — no se revela la existencia de recursos ajenos.

Variables de entorno, nunca credenciales en código.

Las claves de Supabase, las URLs de conexión a la base de datos y cualquier secreto se almacenan en variables de entorno. En desarrollo local, en un archivo .env que está en .gitignore desde el commit inicial. En producción, en Cloud Run secrets o en Secret Manager de Google Cloud. El repositorio nunca contiene credenciales, ni en ramas de feature ni en el historial de git.

La regla operativa para el desarrollador: si alguna vez hacés git add de un archivo que contiene una clave, rotá la clave antes de hacer el push, aunque el repositorio sea privado.

Capa 4 — Frontend

El frontend no es una capa de seguridad.

Esta es la regla más importante del frontend: todo lo que el frontend oculta, un usuario técnico puede verlo igualmente. Los controles de acceso del frontend (no mostrar el botón de crear al propietario, no mostrar el módulo de sanidad al administrador) son controles de UX, no de seguridad. La seguridad real está en el backend y en la base de datos.

Dicho esto, el frontend implementa estos controles porque mejoran la experiencia y porque un sistema que muestra botones que después retornan 403 es confuso y no profesional.

Tokens almacenados en memoria, no en localStorage.

El access token de Supabase Auth se almacena en memoria del cliente, no en localStorage ni en cookies sin HttpOnly. Un script de terceros inyectado en la página no puede leer el token desde localStorage. El SDK de Supabase maneja esto correctamente por defecto — no sobrescribir ese comportamiento.

Sanitización de inputs en formularios.

Los formularios usan React Hook Form con validación Zod. Los campos de texto tienen límites de longitud. Los campos numéricos aceptan solo números. Los campos de fecha rechazan fechas futuras en el cliente (aunque el backend también lo valida). La sanitización del cliente es para UX — la del backend es la que importa para seguridad.

Content Security Policy.

Firebase Hosting permite configurar headers HTTP. La aplicación define un CSP que permite scripts solo desde el propio dominio y desde los CDNs de confianza (Supabase, Firebase). Esto previene XSS en caso de que alguna entrada de usuario logre inyectar contenido en el DOM.

```
Content-Security-Policy: default-src 'self';
script-src 'self' https://*.supabase.co;
connect-src 'self' https://*.supabase.co https://*.googleapis.com;
style-src 'self' 'unsafe-inline';
```

No hay rutas accesibles sin autenticación excepto login y registro.

El router de React implementa un guard que redirige a login si no hay sesión activa. Las rutas protegidas no renderizan nada hasta que la sesión esté validada — no un flash de contenido que después desaparece.

Capa 5 — Infraestructura y deploy

Secrets fuera del repositorio y fuera del código.

Como se definió antes: variables de entorno en desarrollo local, Cloud Run secrets o Secret Manager en producción. La rotación de claves no requiere un redeploy si se usan secrets montados como variables de entorno.

HTTPS obligatorio en todos los endpoints.

Firebase Hosting fuerza HTTPS automáticamente. Cloud Run solo expone HTTPS. No hay comunicación HTTP entre el frontend y el backend en ningún entorno, incluyendo desarrollo con docker-compose (en local se acepta HTTP entre servicios internos de Docker pero no desde el browser).

El backend nunca es accesible directamente desde internet sin autenticación.

Todos los endpoints de la API requieren un Bearer token válido excepto los endpoints de health check. No hay endpoints de debug, de status de base de datos ni de información del sistema expuestos públicamente.

Logs sin datos sensibles.

Los logs de Cloud Logging registran: método HTTP, endpoint, código de respuesta, tiempo de procesamiento, user_id (no el email ni el nombre), establecimiento_id. Los logs nunca incluyen: passwords, tokens, datos de animales, precios, nombres de proveedores ni ningún dato de negocio. Si necesitas depurar un problema específico, usás el ID del evento o del animal para buscarlo en la base de datos directamente, no en los logs.

Backups automáticos de la base de datos.

Supabase realiza backups automáticos diarios en el plan Pro. El esquema de retención es de 7 días en el plan Pro y 30 días en el plan Team. Adicionalmente, una vez por semana se ejecuta un `pg_dump` manual que se almacena en Google Cloud Storage con retención de 90 días. El procedimiento de restore se documenta y se prueba antes del lanzamiento — un backup que nunca se probó no es un backup.

Separación de entornos.

Hay tres entornos: desarrollo local (Docker Compose + Supabase local), staging (Supabase proyecto separado + Cloud Run staging) y producción. Los datos de producción nunca se copian a desarrollo ni a staging. Si se necesitan datos reales para depurar, se trabaja directamente en producción con acceso restringido y auditado.

Controles específicos por regla de negocio

Algunas reglas de negocio del documento de reglas tienen implicancias de seguridad directas que merecen mención explícita.

RN-01 (animales nunca se eliminan) se implementa con ausencia de endpoint `DELETE` en animales y con políticas RLS que no incluyen `DELETE`. No es solo una regla de negocio — es un control de integridad que previene la pérdida de trazabilidad legal.

RN-06 (carencia activa bloquea venta) se implementa en el servicio de backend antes de confirmar cualquier egreso. El frontend muestra el bloqueo pero la validación real ocurre en el servidor. Un cliente que manipule el frontend para saltar la advertencia igual recibe un 400 del backend.

RN-15 (eventos inmutables) se implementa con la rule de PostgreSQL que rechaza updates y con ausencia de endpoint `PUT` o `PATCH` en eventos. La corrección de errores solo es posible mediante un nuevo evento de corrección.

RN-18 (no eventos con fecha futura) se implementa con el constraint `CHECK` en la base de datos. Aunque el backend valide la fecha, el constraint es la garantía final — no puede ser ignorado por ningún bug de aplicación.

Checklist de seguridad antes del lanzamiento

Este checklist se completa durante el Sprint 6 antes de abrir el sistema a clientes reales.

Verificar que RLS está activo en todas las tablas de dominio y que ninguna política permite acceso entre establecimientos. Verificar que ningún endpoint acepta `establecimiento_id` como parámetro externo. Verificar que los tres roles tienen los permisos correctos y que no pueden

escalar privilegios. Verificar que un token válido de un establecimiento no puede acceder a datos de otro, probando manualmente con dos cuentas distintas. Verificar que el historial de git no contiene credenciales (usar git-secrets o trufflehog). Verificar que los logs no contienen datos sensibles de negocio. Verificar que el procedimiento de restore de backup funciona en un entorno de prueba. Verificar que los endpoints de lista tienen paginación y que el límite máximo se respeta en el servidor. Verificar que la importación CSV rechaza archivos malformados, demasiado grandes y con más filas del límite. Verificar que los mensajes de error no revelan la existencia de recursos de otros establecimientos.

Lo que no se implementa en el MVP y por qué

Autenticación de dos factores (2FA). Importante pero no crítico para el MVP con una base de usuarios pequeña y conocida. Se agrega en V2 cuando el sistema tenga usuarios externos que no conocés personalmente.

Auditoría de accesos con pantalla visible. El dato de quién registró cada evento existe en la base desde el día uno (RN-23). La pantalla de auditoría que muestra el historial de accesos y cambios es V2.

Cifrado de datos en reposo a nivel de aplicación. Supabase cifra los datos en reposo a nivel de infraestructura. El cifrado adicional a nivel de aplicación (cifrar campos específicos antes de guardarlos) agrega complejidad sin beneficio real para este caso de uso.

Penetration testing formal. Se hace antes de escalar a más de 50 establecimientos, no antes del MVP con clientes beta controlados.

WAF (Web Application Firewall). Cloud Armor se activa en V2 cuando el sistema tenga tráfico público real. En el MVP con clientes conocidos el rate limiting de FastAPI es suficiente.